

Classification - Part 1

K-Nearest Neighbors and Decision Trees

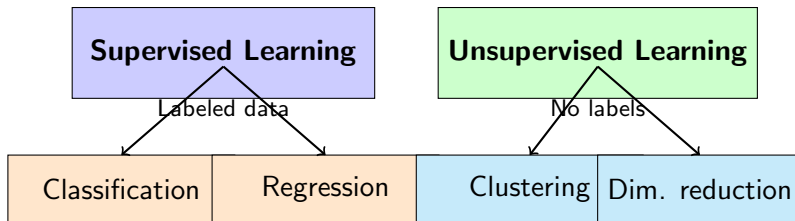
Meyssem Bouzaïen

Academic Year: 2025 - 2026

Course Outline

- 1 What is Classification?
- 2 K-Nearest Neighbors (KNN)
- 3 Decision Trees
- 4 Evaluation Metrics
- 5 Complete Practical Example
- 6 Practical Advice

Reminder: Types of Learning



Today

We focus on **Classification**

What is classification?

Classification consists of **predicting a category** (class) for new observations, based on labeled examples.

Input (Features):

- Measurable characteristics
- Descriptive variables
- Example: size, weight, color...

Output (Class):

- Predicted category
- Discrete variable
- Example: cat, dog, bird

Classification Examples

Application	Features (X)	Classes (y)
Email spam	Words, sender, links	Spam / Not spam
Medical diagnosis	Symptoms, test results	Sick / Healthy
Image recognition	Image pixels	Cat / Dog / Bird
Bank credit	Income, age, history	Approved / Denied
Sentiment analysis	Text of a review	Positive / Negative / Neutral
Flower recognition	Sepal/petal size	Setosa / Versicolor / Virginica

Classification is everywhere!

Binary vs Multi-class Classification

Binary Classification

Only 2 classes

- Yes / No
- Spam / Ham
- Sick / Healthy
- Positive / Negative

Multi-class Classification

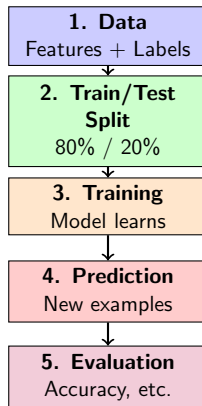
More than 2 classes

- 3 types of Iris flowers
- 10 digits (0-9)
- 26 letters (A-Z)
- 1000 object categories

Important

Today, we'll see algorithms that work for both!

The Classification Process



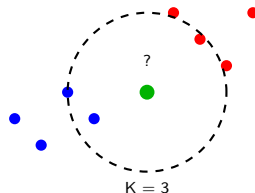
KNN: The Simplest Algorithm

Core Idea

“Tell me who your neighbors are, and I’ll tell you who you are”

Principle:

- 1 Find the K nearest neighbors
- 2 Look at their class
- 3 Majority vote
- 4 Assign the winning class



KNN: How Does It Work? (1/2)

Step 1: Compute the Distance

For each point in the dataset, compute its distance to the new point.

Euclidean distance (most common):

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Example

Point A = (1, 2) and Point B = (4, 6)

$$d = \sqrt{(4 - 1)^2 + (6 - 2)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

KNN: How Does It Work? (2/2)

Step 2: Sort by Distance

Keep the K closest points

Step 3: Majority Vote

Count the classes among the K neighbors

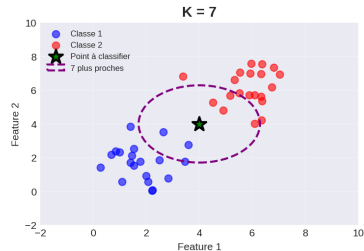
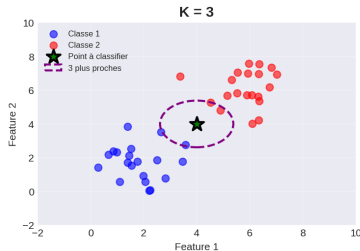
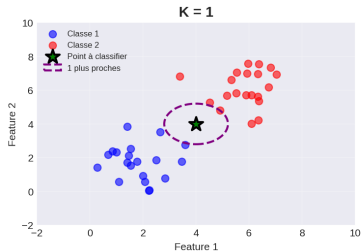
Example with $K=5$

- Neighbor 1: Class A (distance: 1.2)
- Neighbor 2: Class A (distance: 1.5)
- Neighbor 3: Class B (distance: 1.8)
- Neighbor 4: Class A (distance: 2.1)
- Neighbor 5: Class B (distance: 2.3)

Vote: A=3, B=2 → Predicted class = A

The Parameter K: Impact

K-Nearest Neighbors: Impact du paramètre K



Advice

- K too small → overfitting (noise)
- K too large → underfitting (loses detail)
- Common values: $K = 3, 5, 7$ (odd numbers to avoid ties)

KNN with Scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load the data
iris = load_iris()
X = iris.data
y = iris.target

# 2. Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# 3. Create the KNN model
knn = KNeighborsClassifier(n_neighbors=5)

# 4. Train
knn.fit(X_train, y_train)

# 5. Predict
y_pred = knn.predict(X_test)

# 6. Evaluate
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

KNN: Advantages and Disadvantages

Advantages

- Simple to understand
- No training step
- Works well for small datasets
- No assumptions about the data
- Naturally multi-class

Disadvantages

- Slow for large datasets
- Sensitive to un-normalized features
- Sensitive to noise
- Need to choose K
- Memory-intensive

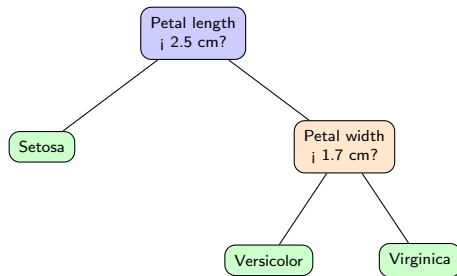
When to Use KNN?

Small datasets, few features, need for simplicity

Decision Trees: Intuition

The Idea

A decision tree asks a series of **questions** about the features to arrive at a decision.



Like a decision flowchart!

How Do We Build a Tree?

Algorithm (simplified)

- ① Choose the best question to ask
- ② Split the data according to that question
- ③ Repeat for each branch
- ④ Stop when:
 - All examples belong to the same class
 - Maximum depth is reached
 - Minimum number of examples is reached

Key Question

How do we choose the **best** question?

→ The one that best separates the classes

Split Criterion: Gini and Entropy

Impurity Measures

We want **pure** groups (a single class per group)

Gini Index:

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

- Gini = 0 → perfectly pure
- Gini = 0.5 → 50/50 mix

Entropy:

$$H = - \sum_{i=1}^n p_i \log_2(p_i)$$

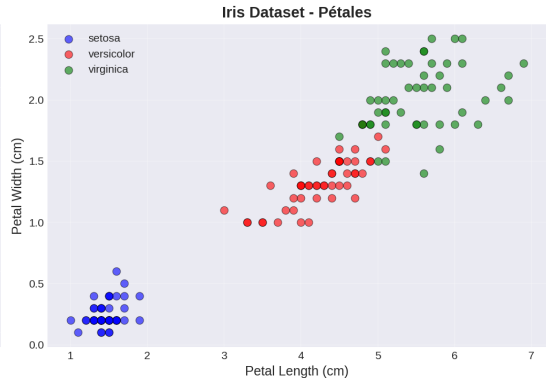
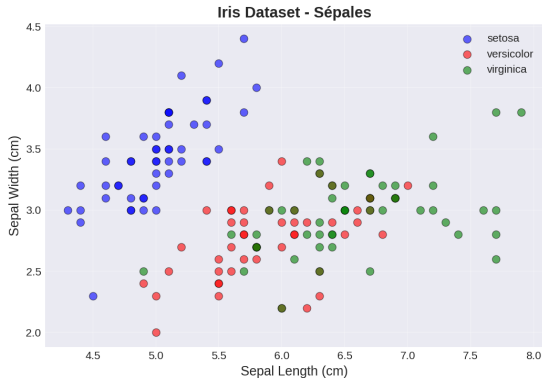
- H = 0 → perfectly pure
- High H → disorder

Example

Group with 10 blue and 0 red → Gini = 0 (pure!)

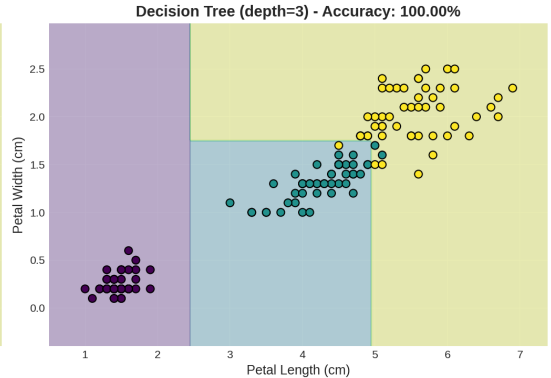
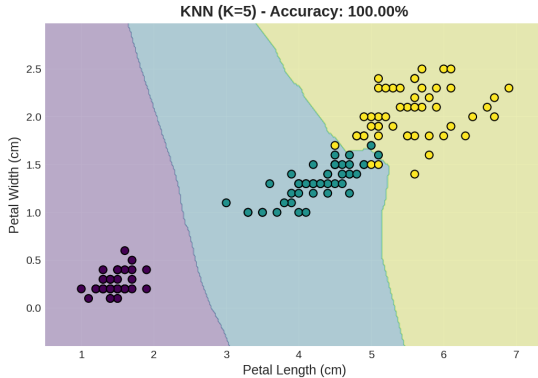
Group with 5 blue and 5 red → Gini = 0.5 (impure)

Visual Example: Iris Dataset



Petals separate the 3 species better than sepals!

Decision Boundaries: KNN vs Tree



KNN: Smooth boundaries — **Tree:** Rectangular boundaries

Decision Trees with Scikit-learn

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Load the data
iris = load_iris()
X = iris.data
y = iris.target

# 2. Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# 3. Create the decision tree
tree = DecisionTreeClassifier(
    max_depth=3,          # Maximum depth
    min_samples_split=2 # Min examples to split
)

# 4. Train
tree.fit(X_train, y_train)

# 5. Predict
y_pred = tree.predict(X_test)

# 6. Evaluate
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

Controlling Tree Complexity

- **max_depth**: Maximum depth of the tree
 - Too large → overfitting
 - Too small → underfitting
- **min_samples_split**: Minimum number of examples to split a node
 - Larger → simpler tree
- **min_samples_leaf**: Minimum number of examples in a leaf
 - Avoids leaves with too little data
- **criterion**: 'gini' or 'entropy'
 - In practice, little difference

Trees: Advantages and Disadvantages

Advantages

- Easy to understand and visualize
- No need for normalization
- Handles numeric and categorical features
- Handles missing values
- Fast to train and predict
- Shows feature importance

Disadvantages

- Tendency to overfit
- Unstable (small changes → different tree)
- Bias toward dominant features
- Less accurate than other methods

Solution

Random Forests (next class) combine several trees to be more robust!

Why Evaluate?

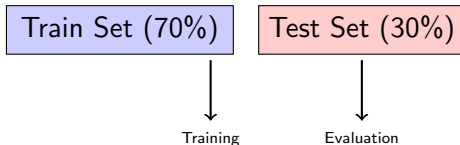
Fundamental Question

How do we know if our model is **good**?

Golden Rule

NEVER evaluate on the training data!

→ Risk of overestimating performance



Accuracy

Definition

Percentage of correct predictions

$$Accuracy = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

Example

Out of 100 flowers:

- 95 correctly classified
- 5 misclassified

$$Accuracy = \frac{95}{100} = 0.95 = 95\%$$

Warning!

Accuracy can be misleading if classes are imbalanced!

The Problem of Imbalanced Classes

Example: Fraud Detection

Dataset: 1000 transactions

- 990 normal
- 10 fraudulent

Naive model: Predict "Normal" for everything

→ Accuracy = 99%!

But: 0% of fraud detected!

Solution

Use other metrics: Precision, Recall, F1-Score

Confusion Matrix

Definition

A table showing predictions vs actual values

		Prediction	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

- **TP** (True Positive): Predicted positive, correct
- **TN** (True Negative): Predicted negative, correct
- **FP** (False Positive): Predicted positive, wrong (Type I Error)
- **FN** (False Negative): Predicted negative, wrong (Type II Error)

Confusion Matrix Example

Classifying 100 emails

		Prediction	
		Spam	Ham
Actual	Spam	35	5
	Ham	3	57

- TP = 35 (spam correctly detected)
- TN = 57 (ham correctly classified)
- FP = 3 (ham classified as spam)
- FN = 5 (spam not detected)

$$\text{Accuracy} = \frac{35+57}{100} = 92\%$$

Precision and Recall

Precision

Among the positive predictions, how many are correct?

$$\text{Precision} = \frac{TP}{TP+FP}$$

"When I say spam, is it really spam?"

Recall

Among the actual positives, how many are detected?

$$\text{Recall} = \frac{TP}{TP+FN}$$

"Do I detect all the spam?"

Previous Example

$$\text{Precision} = \frac{35}{35+3} = 0.92 = 92\%$$

$$\text{Recall} = \frac{35}{35+5} = 0.875 = 87.5\%$$

F1-Score

Problem

Precision and Recall can conflict!

Trade-off

- Increasing Precision → decreases Recall
- Increasing Recall → decreases Precision

F1-Score: Harmonic Mean

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- F1-Score balances Precision and Recall
- Useful when classes are imbalanced

Computing Metrics with Scikit-learn

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report

# Predictions
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

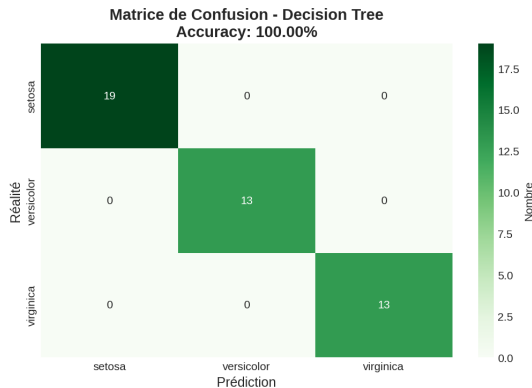
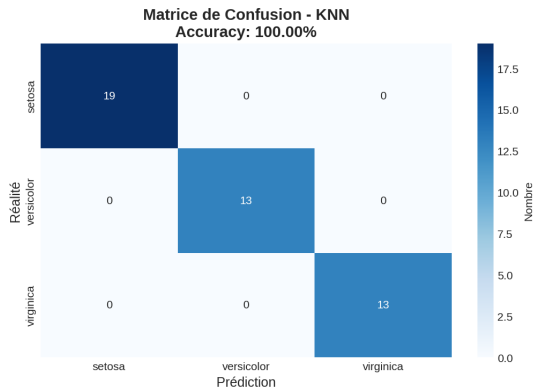
# Precision, Recall, F1
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-Score: {f1:.2f}")

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)

# Full report
print(classification_report(y_test, y_pred))
```

Confusion Matrices: KNN vs Tree



Both models achieve 100% accuracy on the Iris test set!

Iris Dataset: Reminder

```
from sklearn.datasets import load_iris
import pandas as pd

# Load
iris = load_iris()

# Create a DataFrame for visualization
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

print(df.head())
print("\nClasses:", iris.target_names)
print("Features:", iris.feature_names)
```

- 150 flowers
- 4 features (sepal/petal length and width)
- 3 classes (Setosa, Versicolor, Virginica)

KNN vs Tree Comparison (1/2)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load and split
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42)

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)

# Decision Tree
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)
y_pred_tree = tree.predict(X_test)
```


KNN vs Tree Comparison (2/2)

```
# Evaluate KNN
print("=== KNN ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred_knn):.2f}")
print(classification_report(y_test, y_pred_knn,
                           target_names=iris.target_names))

# Evaluate Tree
print("\n=== Decision Tree ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred_tree):.2f}")
print(classification_report(y_test, y_pred_tree,
                           target_names=iris.target_names))
```

Typical results:

- KNN: 95-98% accuracy
- Tree: 93-96% accuracy

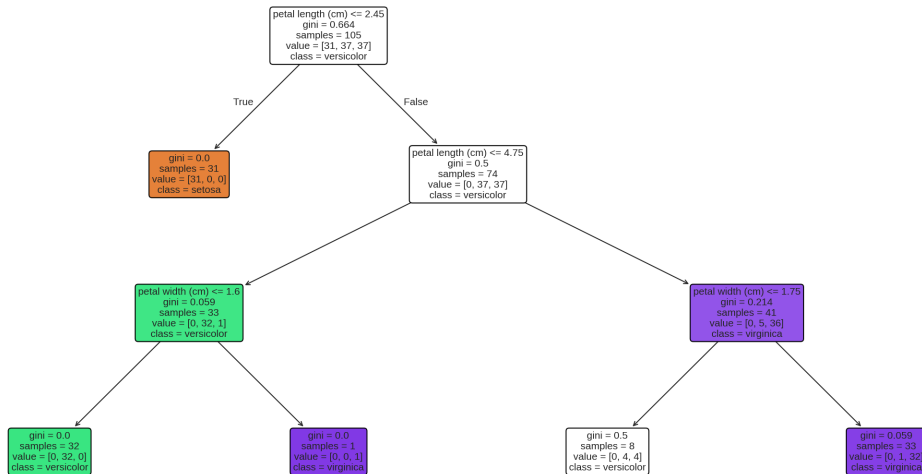
Visualizing the Decision Tree

```
1 from sklearn.tree import plot_tree
2 import matplotlib.pyplot as plt
3
4 # Create a large figure
5 plt.figure(figsize=(20,10))
6
7 # Visualize the tree
8 plot_tree(tree,
9           feature_names=iris.feature_names,
10          class_names=iris.target_names,
11          filled=True,
12          rounded=True,
13          fontsize=10)
14
15 plt.title("Decision Tree - Iris Dataset")
16 plt.savefig('decision_tree_iris.png', dpi=150, bbox_inches='tight')
17 plt.show()
```

Helps us understand how the tree makes its decisions!

The Visualized Decision Tree

Arbre de Décision - Dataset Iris (4 features)



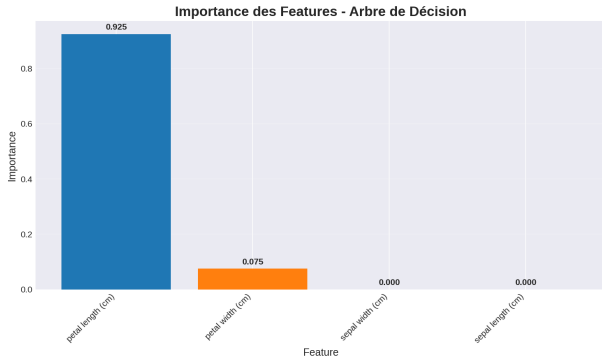
Feature Importance

```
import numpy as np
import matplotlib.pyplot as plt

# Get importance values
importances = tree.feature_importances_
features = iris.feature_names

# Sort
indices = np.argsort(
    importances)[::-1]

# Visualize
plt.bar(range(len(importances)),
        importances[indices])
plt.xticks(range(len(importances)),
           [features[i] for i in indices])
plt.title("Feature Importance")
plt.show()
```



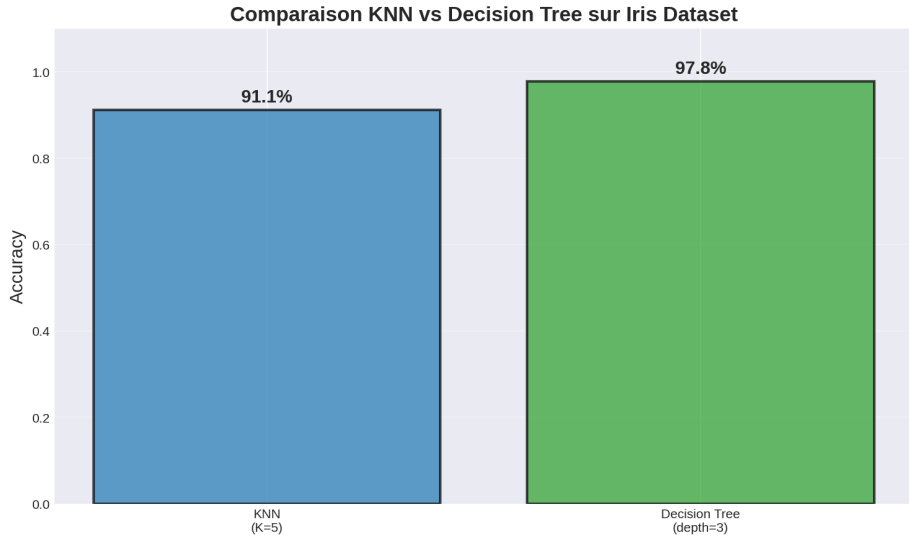
KNN vs Trees: When to Use What?

Use KNN if:	Use Trees if:
Small/medium dataset	Medium/large dataset
Few features (≤ 20)	Many features
Well-normalized data	No need to normalize
Complex non-linear relationships	Need for interpretability
Training time not critical	Prediction time matters
No need for explanation	Need to understand the rules

In practice

Try both and compare!

Final Comparison: Performance



Best Practices

- 1 **Always** split train/test
- 2 **Normalize** the data for KNN
- 3 **Test several values** of K (KNN) or max_depth (Trees)
- 4 **Visualize** the results (confusion matrix)
- 5 **Don't** rely on accuracy alone
- 6 **Check** for overfitting (train vs test accuracy)
- 7 **Understand** the model's errors

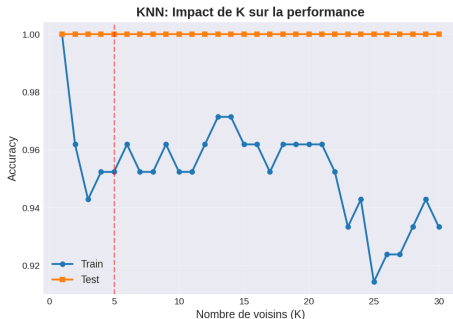
Recommended Workflow

Exploration → Preparation → Several models → Comparison → Best model

Avoiding Overfitting

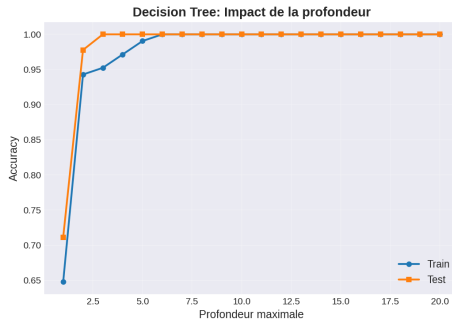
With KNN:

- Increase K
- Normalize the features
- Reduce the number of features
- More training data



With Trees:

- Limit max_depth
- Increase min_samples_split
- Increase min_samples_leaf
- Pruning



Your Turn!

- 1 Load the Iris dataset
- 2 Explore the data (shape, describe, visualization)
- 3 Split into train/test (70/30)
- 4 Train a KNN with $K=3$
- 5 Train a tree with `max_depth=3`
- 6 Compare both models (accuracy, confusion matrix)
- 7 Test different values of K and `max_depth`
- 8 Find the best parameters
- 9 Visualize the final tree
- 10 Interpret the results

What You've Learned

- **Classification:** predicting categories
- **KNN:** vote of the K nearest neighbors
- **Decision trees:** a series of questions
- **Metrics:** accuracy, precision, recall, F1
- **Confusion matrix:** visualizing errors
- **Best practices:** normalization, validation, avoiding overfitting